Lets start a project for ai based regression testing application. Starts with building a website or webapp with python. Use python and uv for all related techs. I have foundry local installed in my laptop with LLM model - qwen2.5-0.5b and VLM model -- qwen3-vl-2b-instruct which are already downloaded. Keep regression testing app and demo website for testing separate. Start with planning first. After approval start with code.I already have this structure of the project :

ai-regression-tests/ ├── .python-version ├── pyproject.toml ├── README.md ├── ai/ │ ├── **init**.py │ ├── foundry_client.py │ └── prompts.py ├── reports/ ├── screenshots/ └── tests/ ├── **init**.py ├── test_ai_visual_regression.py └── test_web_regression.py

Great starting point. Here's a focused plan for an **AI-based regression testing application**, built in Python with `uv`, using your local Foundry Local models. Foundry Local exposes an OpenAI-compatible API, so the app can use the standard OpenAI Python SDK against a locally discovered endpoint. [1] [2]

Project goals

Build two independent applications inside one repository:

1. **AI Regression Testing App** — captures website screenshots, compares them across runs, and uses local AI to classify visual differences and explain likely regressions.
2. **Demo Website** — a small, deliberately changeable Python web app used only as the target for regression tests.

This separation prevents test-tooling code from becoming entangled with the application being tested.

Proposed architecture

```
ai-regression-tests/
├── .python-version
├── pyproject.toml
├── README.md
```

```
├── .env.example
├── ai/
│   ├── __init__.py
│   ├── foundry_client.py
│   └── prompts.py
├── app/                      # Regression-testing web app
│   ├── __init__.py
│   ├── main.py
│   ├── routes/
│   ├── services/
│   │   ├── screenshot.py
│   │   ├── diff.py
│   │   └── ai_analysis.py
│   ├── models/
│   └── static/
├── demo_site/                # Separate demo website under test
│   ├── __init__.py
│   ├── main.py
│   ├── routes/
│   ├── templates/
│   └── static/
├── reports/
├── screenshots/
│   ├── baseline/
│   ├── current/
│   └── diff/
├── tests/
│   ├── __init__.py
│   ├── test_ai_visual_regression.py
│   └── test_web_regression.py
└── scripts/
    ├── run_regression.py
    └── approve_baseline.py
```

Core workflow

1. Start the demo website locally.
2. Run the regression suite with Playwright.
3. Capture screenshots of selected pages, components, and states.
4. Compare current screenshots with approved baselines using pixel/image diffing.
5. Send meaningful visual differences to `qwen3-vl-2b-instruct` for an AI explanation.
6. Use `qwen2.5-0.5b` for lightweight text tasks, such as summarizing failures, classifying severity, or generating concise test reports.
7. Produce an HTML report showing baseline, current, diff, confidence, and AI analysis.
8. Let you explicitly approve a new screenshot as the new baseline.

Technology choices

Area	Choice	Reason
Package/environment manager	`uv`	Fast dependency and virtual-environment management
Web framework	FastAPI + Uvicorn	Lightweight Python web apps for both the testing UI and demo site
Browser automation	Playwright	Reliable screenshots and browser-driven regression checks
Image diffing	OpenCV or Pillow + scikit-image	Deterministic pixel/structural comparison before AI analysis
AI runtime	Foundry Local	Runs your downloaded models locally
AI client	`openai` SDK	Foundry Local supports OpenAI-compatible APIs. [1] [2]
Text model	`qwen2.5-0.5b`	Fast local model for summaries and report text
Vision model	`qwen3-vl-2b-instruct`	Analyzes screenshots and visual regressions
Testing	pytest	Standard Python test runner
Reports	Jinja2-generated HTML	Easy local review of regressions

AI responsibilities

The AI should **assist**, not replace, deterministic checks.

- **Pixel/image diff first:** Detect whether something changed and where.
- **Vision model second:** Explain whether the change looks like a real UI regression, an acceptable rendering variation, or a content update.
- **Text model third:** Turn findings into concise, actionable regression notes.

Example output:

```
{
  "page": "/checkout",
  "visual_diff_percent": 2.4,
  "ai_classification": "likely_regression",
  "severity": "high",
  "summary": "The primary payment button appears shifted and partially obscured.",
  "recommended_action": "Inspect checkout layout changes and update baseline only af
}
```

Foundry Local integration

`ai/foundry_client.py` should:

- Use `FoundryLocalManager` to discover the local endpoint dynamically rather than hardcoding a port.

- Create an OpenAI-compatible client.
- Expose two methods:
  - `analyze_image(image_path, prompt)`
  - `summarize_text(prompt, context)`
- Read model names from environment variables:

```
FOUNDRY_TEXT_MODEL=qwen2.5-0.5b
FOUNDRY_VISION_MODEL=qwen3-vl-2b-instruct
DEMO_SITE_BASE_URL=http://127.0.0.1:8001
REGRESSION_APP_URL=http://127.0.0.1:8000
```

Foundry Local's Python SDK can start or connect to the local service and supply the model ID and endpoint to the OpenAI client.[2] [3]

MVP scope

Phase 1 — Foundation

- Configure `uv`, dependencies, settings, and folder structure.
- Implement Foundry Local client with health checks for both models.
- Create the demo website with 3–4 pages and intentional UI variations.
- Add Playwright screenshot capture.

Phase 2 — Regression engine

- Implement baseline storage and screenshot naming.
- Add pixel-level diffing.
- Generate diff images and difference metrics.
- Add baseline approval workflow.

Phase 3 — AI analysis

- Send cropped or full-page screenshots to the vision model.
- Ask for structured JSON output: classification, severity, explanation, and recommended action.
- Use the text model to create a readable executive summary.
- Add prompt templates in `ai/prompts.py`.

Phase 4 — Web dashboard

- FastAPI dashboard listing test runs.
- Side-by-side baseline/current/diff viewer.
- Run history and pass/fail status.
- Downloadable HTML report.

Phase 5 — Tests and polish

- Unit tests for diffing, configuration, and AI response parsing.
- Integration tests for screenshot capture and report generation.
- README with local setup and demo instructions.

Important design decisions

- **Keep AI optional at runtime.** If Foundry Local is unavailable, tests should still produce deterministic image diffs and reports.
- **Never auto-approve baselines.** A changed screenshot should require explicit human approval.
- **Use structured AI output.** Ask the vision model for JSON and validate it with Pydantic.
- **Make the demo site intentionally unstable.** Include a feature flag or environment variable that changes colors, layout, text, or button placement so regressions can be demonstrated reliably.
- **Store artifacts by run ID.** For example: `screenshots/current/2026-11-05T16-42-00/home.png`.

Suggested dependencies

```
dependencies = [
    "fastapi",
    "uvicorn",
    "jinja2",
    "playwright",
    "pillow",
    "opencv-python",
    "scikit-image",
    "numpy",
    "pydantic",
    "pydantic-settings",
    "openai",
    "foundry-local-sdk",
    "python-multipart",
]

dependency-groups = {
    dev = [
        "pytest",
        "pytest-asyncio",
```

```
        "httpx",
        "ruff",
    ]
}
```

Proposed first code milestone

Once approved, I'll start with:

1. Update `pyproject.toml` for `uv`.
2. Add configuration and Foundry Local client.
3. Build the separate FastAPI demo website.
4. Add Playwright screenshot capture.
5. Implement baseline/current screenshot comparison.
6. Add a minimal HTML regression report.
7. Provide commands to run both apps and the regression suite.

Shall I proceed with **Phase 1: project setup, Foundry Local client, demo website, and screenshot capture**?

⁂

1. https://learn.microsoft.com/en-us/azure/foundry-local/what-is-foundry-local
2. https://learn.microsoft.com/en-us/azure/ai-foundry/foundry-local/how-to/how-to-integrate-with-inference-sdks?pivots=programming-language-python
3. https://learn.microsoft.com/en-us/azure/foundry-local/reference/reference-sdk-legacy
4. https://learn.microsoft.com/en-us/azure/foundry-local/how-to/how-to-integrate-with-inference-sdks
5. https://learn.microsoft.com/fil-ph/azure/foundry-local/reference/reference-rest
6. https://microsoft.github.io/ai-agents-for-beginners/translations/bn/17-creating-local-ai-agents/
7. https://microsoft.github.io/ai-agents-for-beginners/translations/te/17-creating-local-ai-agents/
8. https://microsoft.github.io/ai-agents-for-beginners/17-creating-local-ai-agents/
9. https://microsoft.github.io/ai-agents-for-beginners/translations/hi/17-creating-local-ai-agents/
10. https://github.com/microsoft/foundry-local
11. https://github.com/aaronpowell/Foundry-Local/blob/main/docs/README.md
12. https://docs.rs/agent-framework-foundry-local/latest/agent_framework_foundry_local/
13. https://learn.microsoft.com/en-in/azure/ai-foundry/foundry-local/how-to/how-to-integrate-with-inference-sdks
14. http://www.learn.microsoft.com/ru-ru/azure/foundry-local/how-to/how-to-integrate-with-inference-sdks
15. https://learn.microsoft.com/en-za/azure/foundry-local/how-to/how-to-integrate-with-inference-sdks?context=/windows-server/context/windows-server-ai
16. https://learn.microsoft.com/en-us/azure/foundry/how-to/develop/sdk-overview?pivots=programming-language-python
17. https://ai.azure.com/catalog/models/qwen-qwen3-vl-2b-thinking

18. https://github.com/thangchung/microsoft-Foundry-Local
19. https://github.com/microsoft/Foundry-Local/blob/main/sdk/python/README.md
20. https://github.com/microsoft-foundry/Foundry-Local-Lab/blob/main/labs/part3-sdk-and-apis.md
21. https://github.com/microsoft/foundry-local/blob/main/README.md
22. https://github.com/microsoft/foundry-local/issues/741
23. https://learn.microsoft.com/en-in/azure/foundry-local/reference/reference-rest
24. https://www.learn.microsoft.com/en-us/azure/ai-foundry/foundry-local/how-to/how-to-integrate-with-inference-sdks
25. https://learn.microsoft.com/en-gb/azure/ai-foundry/foundry-local/reference/reference-rest
26. https://devblogs.microsoft.com/foundry/foundry-local-ga/
27. https://learn.microsoft.com/nb-no/azure/foundry-local/how-to/how-to-use-native-chat-completions
28. https://learn.microsoft.com/en-ie/azure/foundry-local/what-is-foundry-local